INFLOW - Midterm Progress Report

Rohan Upadhyay | CSCI 412: Senior Seminar II | March 13, 2026

GitHub: https://github.com/RyanX5/INFLOW

---

1. Project Overview

INFLOW is an agent-based simulation designed to study how information and misinformation propagate through social networks. The system models individual users as agents with psychological traits that influence how they receive, believe, and share information. By combining network science with behavioral modeling, INFLOW simulates how different types of content spread across a population.

The central goal of the project is to explore the dynamics that allow misinformation to spread rapidly in online environments. In particular, the simulation examines how emotional intensity, individual bias, skepticism, and network structure interact to shape information diffusion.

Agents exist within a network topology that approximates real social networks. Information items are introduced into the network from origin nodes and propagate through agent interactions. Each agent updates its belief state when receiving information and may choose to share it with neighbors based on probabilistic behavioral rules.

The project demonstrates core computer science concepts including agent-based modeling, network theory, probabilistic simulation, and data visualization. In addition to the simulation engine, a web-based visualization system allows propagation patterns to be explored interactively.

---

2. Implementation Status

Since the initial proposal, the core simulation framework has been implemented and is now capable of running full information propagation experiments.

Completed Modules

Module	Status	Description
`simulation/network.py`	Complete	Network generation and topology construction
`simulation/agent.py`	Complete	Agent behavioral model including belief updating
`simulation/information.py`	Complete	Information item representation
`simulation/engine.py`	Complete	Simulation engine controlling propagation
`main.py`	Complete	Simulation entry point and experiment setup

Module	Status	Description
`visualization/`	Complete	React + D3 visualization frontend

Repository: https://github.com/RyanX5/INFLOW

3. Timeline Comparison

Original proposed timeline:

- Week 1--2: Simulation framework and agent model
- Week 3--4: Network generation and propagation logic
- Week 5--6: Information model and belief updates
- Week 7--8: Data logging
- Week 9: Visualization
- Week 10: Midterm report

Current status: All milestones through Week 9 have been completed.

---

4. System Architecture

INFLOW consists of three subsystems:

- `simulation/` --- backend simulation engine
- `visualization/` --- React + D3 frontend

- `output/` --- generated CSV datasets

Pipeline:

Network Generation → Agent Initialization → Information Injection → Simulation Execution → Data Logging → Visualization

Example simulation execution:

```
from simulation.network import Network
from simulation.engine import SimulationEngine
from simulation.information import Information

network = Network(num_agents=100)

engine = SimulationEngine(network)

true_info = Information(truth_value=0.9, emotional_intensity=0.2)
false_info = Information(truth_value=0.1, emotional_intensity=0.9)

engine.inject_information(true_info)
engine.inject_information(false_info)

engine.run(steps=20)
```

---

5. Key Implementations

Agent Behavioral Model

Agents maintain behavioral parameters:

belief ∈ [0,1]
bias ∈ [0,1]
skepticism ∈ [0,1]
trust_radius ∈ [0,1]

Belief update rule:

$$\Delta b = \frac{T - b}{1 + s}(1 - \beta)$$

Where:

- b is current belief
- T truth value
- s skepticism
- β bias

Updated belief:

$$b' = clamp(b + \Delta b, 0, 1)$$

Example implementation:

```
def update_belief(self, truth_value):
    delta = ((truth_value - self.belief) / (1 + self.skepticism)) * (1 - self
    self.belief = max(0, min(1, self.belief + delta))
```

Information Representation

Information items represent pieces of content that propagate through the network.
Each item contains:

- truth_value ∈ [0,1]
- emotional_intensity ∈ [0,1]
- origin_node
- spread_count

Example structure:

```
class InfoItem:
    def __init__(self, item_id, truth_value, emotional_intensity, origin_node
        self.item_id = item_id
        self.truth_value = truth_value
        self.emotional_intensity = emotional_intensity
        self.origin_node = origin_node
        self.spread_count = 0
```

Information items are injected into the simulation by the engine and begin propagating from the origin node.

Simulation Engine

The simulation engine manages the entire propagation process, executing discrete time steps and coordinating interactions between agents.

Key responsibilities include:

- Injecting information into the network
- Executing time steps
- Determining information sharing decisions
- Logging propagation statistics
- Exporting results for visualization

Example simulation loop from `simulation/engine.py`:

```
def run(self, num_steps: int):

    print(f"\nStarting simulation: {num_steps} steps, "
          f"{len(self.agents)} agents, {len(self.info_items)} info item(s)\n"

    for _ in range(num_steps):
        self.step += 1
        self._step()
        self._log_step()
        self._print_step_summary()

    print(f"\nSimulation complete after {self.step} steps.")
```

During each step, agents that possess a piece of information may share it with their neighbors.

Example propagation logic:

```
for agent in holders:
    neighbors = get_neighbors(self.graph, agent.agent_id)

    for neighbor_id in neighbors:
        neighbor = self.agents[neighbor_id]

        if item.item_id in neighbor.received_info:
            continue

        p = self._sharing_probability(agent, neighbor, item)

        if random.random() < p:
            neighbor.receive_information(item)
            agent.shared_info.append(item.item_id)
            item.spread_count += 1
```

Sharing Probability Model

The probability that an agent shares information depends on several factors:

- emotional intensity of the information

- alignment between agent belief and information truth value
- trust radius of the agent

Implementation excerpt:

```
def _sharing_probability(self, agent, neighbor, item):

    alignment = 1.0 - abs(agent.belief - item.truth_value)

    prob = (
        0.4 * item.emotional_intensity +
        0.3 * alignment +
        0.3 * agent.trust_radius
    )

    return max(0.0, min(1.0, prob))
```

This probabilistic rule allows emotionally intense misinformation to spread faster than neutral truthful information, which mirrors patterns observed in real-world social networks.

---

6. Data Logging

Simulation output is recorded in CSV files for analysis and visualization.

Generated datasets include:

- `agent_states.csv`
- `edges.csv`
- `info_items.csv`
- `spread_log.csv`

Example logging structure:

```
self.spread_log.append({
    "step": self.step,
    "item_id": item.item_id,
    "spread_count": item.spread_count,
    "total_agents": len(self.agents)
})
```

These datasets allow propagation dynamics to be analyzed after the simulation completes.

---

7. Visualization System

To better understand propagation behavior, the project includes a web-based visualization built with:

- React
- D3.js
- TypeScript
- Vite

The visualization loads the CSV output files generated by the simulation and renders an interactive network graph.

Features include:

- node-link network visualization
- animated propagation of information
- visual differentiation between information items
- interactive exploration of agent connections

This visualization allows users to observe how clusters form and how information cascades move through the network.

---

8. Demonstration

To verify the functionality of the **INFLOW** simulation, the system was executed and the resulting data outputs were visualized using the frontend interface.

The simulation produces several **CSV files** that log:

- Agent states
- Network edges
- Information items
- Propagation events

These files are then loaded by the visualization system to display the structure of the network and the spread of information across agents.

Video Demonstration

For a full walkthrough of the simulation engine and real-time visualization, view the demonstration video below:

https://www.youtube.com/watch?v=d6kHgAkSpR0

Network & Data Visualization

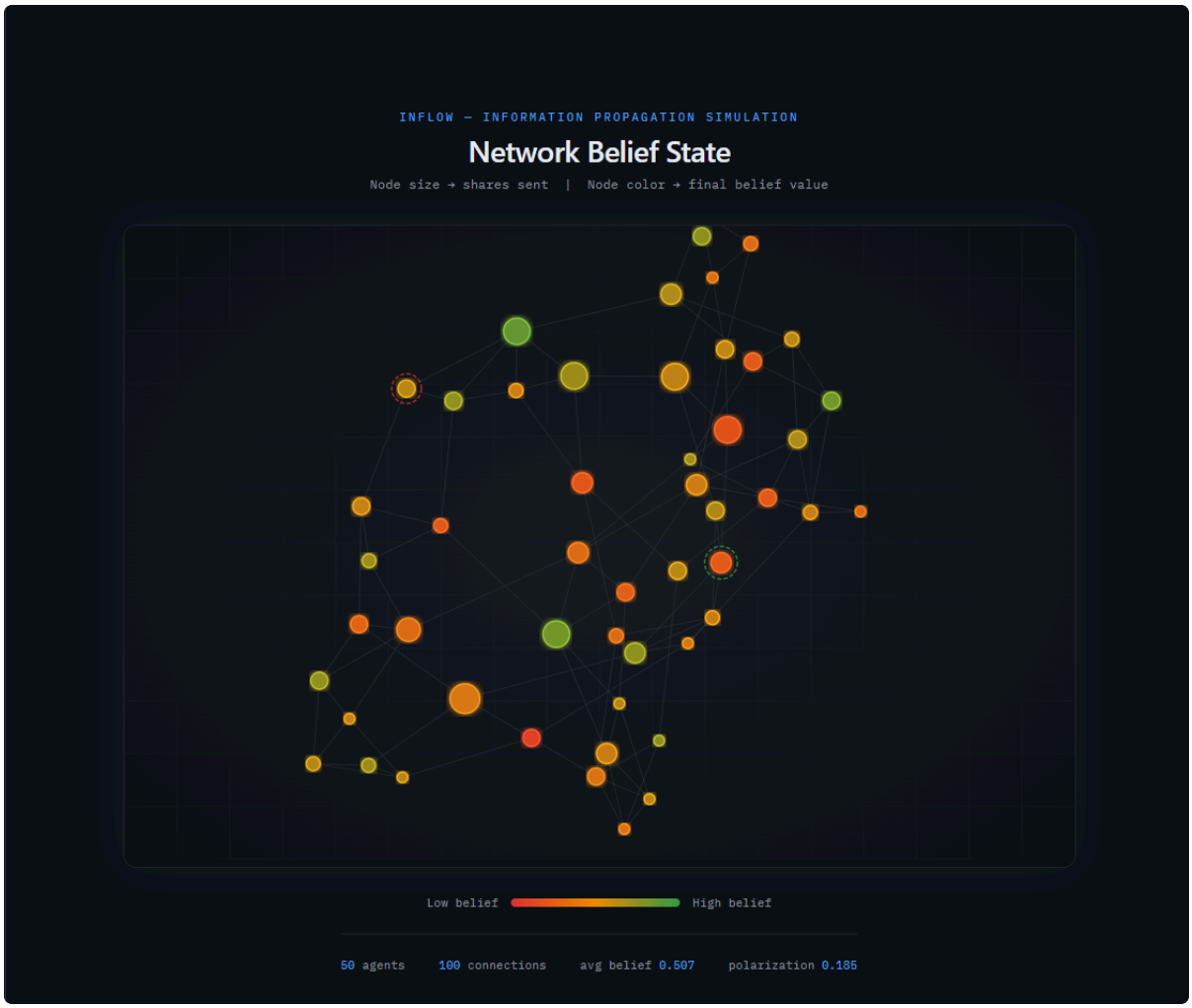

Fig 1: Visualization of the generated agent network. Nodes represent agents and edges represent communication links.

output > spread_log.csv

```csv
step,item_id,spread_count,total_agents,spread_fraction,avg_belief,min_belief,max_belief
1,0,3,50,0.06,0.4033,0.2175,0.7032
1,1,2,50,0.04,0.5598,0.2697,0.85
2,0,5,50,0.1,0.5338,0.2175,0.8388
2,1,5,50,0.1,0.4293,0.0922,0.85
3,0,8,50,0.16,0.5765,0.2175,0.8388
3,1,10,50,0.2,0.4078,0.0922,0.85
4,0,12,50,0.24,0.6221,0.2175,0.8589
4,1,17,50,0.34,0.3978,0.0922,0.85
5,0,19,50,0.38,0.5622,0.1931,0.8589
5,1,27,50,0.54,0.4088,0.0922,0.85
6,0,28,50,0.56,0.4795,0.0989,0.9212
6,1,44,50,0.88,0.4183,0.0846,0.8534
7,0,38,50,0.76,0.4811,0.0989,0.8534
7,1,50,50,1.0,0.4638,0.0846,0.8534
8,0,43,50,0.86,0.4954,0.0989,0.8534
8,1,50,50,1.0,0.4752,0.0846,0.8534
9,0,49,50,0.98,0.5061,0.0989,0.8534
9,1,50,50,1.0,0.507,0.0989,0.8534
10,0,50,50,1.0,0.5075,0.0989,0.8534
10,1,50,50,1.0,0.5075,0.0989,0.8534
11,0,50,50,1.0,0.5075,0.0989,0.8534
11,1,50,50,1.0,0.5075,0.0989,0.8534
12,0,50,50,1.0,0.5075,0.0989,0.8534
12,1,50,50,1.0,0.5075,0.0989,0.8534
13,0,50,50,1.0,0.5075,0.0989,0.8534
13,1,50,50,1.0,0.5075,0.0989,0.8534
14,0,50,50,1.0,0.5075,0.0989,0.8534
14,1,50,50,1.0,0.5075,0.0989,0.8534
15,0,50,50,1.0,0.5075,0.0989,0.8534
15,1,50,50,1.0,0.5075,0.0989,0.8534
```

Fig 2: Example rows from the spread_log.csv file showing recorded information propagation events.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Building 'small_world' network with 50 agents...
  Nodes: 50 | Edges: 100 | Avg degree: 4.0 | Connected: True
  [Step 0] Injected InfoItem(id=0, truth=0.90, emotion=0.20, origin=40) at agent 40
  [Step 0] Injected InfoItem(id=1, truth=0.10, emotion=0.90, origin=7) at agent 7

Starting simulation: 15 steps, 50 agents, 2 info item(s)

  Step   1 | Item 0 | Reached 3/50 agents (6.0%)
  Step   1 | Item 1 | Reached 4/50 agents (8.0%)
  Step   2 | Item 0 | Reached 7/50 agents (14.0%)
  Step   2 | Item 1 | Reached 7/50 agents (14.0%)
  Step   3 | Item 0 | Reached 10/50 agents (20.0%)
  Step   3 | Item 1 | Reached 13/50 agents (26.0%)
  Step   4 | Item 0 | Reached 13/50 agents (26.0%)
  Step   4 | Item 1 | Reached 22/50 agents (44.0%)
  Step   5 | Item 0 | Reached 15/50 agents (30.0%)
  Step   5 | Item 1 | Reached 33/50 agents (66.0%)
  Step   6 | Item 0 | Reached 20/50 agents (40.0%)
  Step   6 | Item 1 | Reached 42/50 agents (84.0%)
  Step   7 | Item 0 | Reached 25/50 agents (50.0%)
  Step   7 | Item 1 | Reached 49/50 agents (98.0%)
  Step   8 | Item 0 | Reached 31/50 agents (62.0%)
  Step   8 | Item 1 | Reached 50/50 agents (100.0%)
  Step   9 | Item 0 | Reached 36/50 agents (72.0%)
  Step   9 | Item 1 | Reached 50/50 agents (100.0%)
  Step  10 | Item 0 | Reached 45/50 agents (90.0%)
  Step  10 | Item 1 | Reached 50/50 agents (100.0%)
  Step  11 | Item 0 | Reached 47/50 agents (94.0%)
  Step  11 | Item 1 | Reached 50/50 agents (100.0%)
  Step  12 | Item 0 | Reached 49/50 agents (98.0%)
  Step  12 | Item 1 | Reached 50/50 agents (100.0%)
  Step  13 | Item 0 | Reached 50/50 agents (100.0%)
  Step  13 | Item 1 | Reached 50/50 agents (100.0%)
  Step  14 | Item 0 | Reached 50/50 agents (100.0%)
  Step  14 | Item 1 | Reached 50/50 agents (100.0%)
  Step  15 | Item 0 | Reached 50/50 agents (100.0%)
  Step  15 | Item 1 | Reached 50/50 agents (100.0%)

Simulation complete after 15 steps.
```

Fig 3: Terminal output of a simulation running with two information injections.

Summary of Results

These demonstrations confirm that the simulation engine successfully:

- **Generates** network structures.
- **Simulates** information diffusion.

- **Records** propagation events for analysis or visualization.

The modular design of the project also allows the simulation and visualization components to operate independently while sharing standardized data outputs.

---

9. Remaining Work

Although the core simulation framework has been implemented, several improvements remain for the final version of the project.

Planned work includes:

- experimentation with different network topologies
- parameter tuning for agent behavioral models
- improved visualization features
- statistical analysis of propagation outcomes
- comparison between true and false information spread patterns

These additions will allow the simulation to provide deeper insights into misinformation dynamics.

---

10. Challenges Encountered

During development, several challenges arose while designing and implementing the simulation.

One challenge involved designing behavioral rules that realistically capture how agents respond to information. Translating abstract social concepts such as bias, skepticism, and emotional influence into mathematical rules required experimentation to ensure that the resulting simulation produced believable propagation patterns.

Another challenge involved integrating the backend simulation with the frontend visualization system. The simulation generates large amounts of data describing agent states and propagation events, which needed to be exported in a format that the React and D3 visualization could efficiently load and render. Establishing a consistent CSV data pipeline between the Python simulation and the visualization frontend required careful coordination of data structures and logging formats.

Addressing these challenges helped refine the architecture of the system and ensured that the simulation components and visualization tools operate reliably together.

---

11. Conclusion

The INFLOW system successfully implements an agent-based model for studying information propagation in social networks. The project integrates concepts from network science, behavioral modeling, and probabilistic simulation to produce a flexible experimental platform.

At the midpoint of the project, the simulation engine, agent models, and visualization system have all been implemented and tested. Initial experiments

demonstrate that the system can generate realistic diffusion patterns and provide insight into how behavioral traits influence information spread.

Future work will focus on expanding experimental scenarios and conducting systematic analyses of misinformation dynamics across different network structures.

---

12. Use of AI Tools and Professional Conduct

During the development of this project, generative AI tools were occasionally used as supplementary resources for brainstorming, formatting assistance, and clarification of programming concepts. These tools were used in a manner similar to consulting documentation or online programming forums.

All core project design decisions, simulation logic, algorithms, and implementation code were written and developed by the author. AI tools were not used to generate the primary architecture or core functionality of the system.

In particular, AI assistance was used for:

- formatting and refining written sections of the report
- clarifying documentation for libraries and frameworks
- minor debugging suggestions during development

All generated suggestions were reviewed, modified when necessary, and integrated manually. The author maintained full responsibility for the design, correctness, and integrity of the final system and report.

This use of AI tools complies with the course guidelines for professional conduct and responsible use of generative technologies.

---

13. References

Barabási, A. (2016). *Network Science*. Cambridge University Press.
https://networksciencebook.com/

Watts, D. J., & Strogatz, S. H. (1998). *Collective dynamics of small-world networks*.
Nature 393.

Vosoughi, S., Roy, D., & Aral, S. (2018). *The spread of true and false news online*. Science. https://doi.org/10.1126/science.aap9559

Newman, M. (2010). *Networks: An Introduction*. Oxford University Press.

D3.js Data Visualization Library
https://d3js.org/

React Documentation
https://react.dev/